

Real-Time Examples of Interface Segregation Principle in C#

In this article, I will discuss **Multiple Real-Time Examples of the Interface Segregation Principle (ISP) in C#**. Please read our previous article discussing the basic concepts of the [Interface Segregation Principle \(ISP\) in C#](#). At the end of this article, you will understand the following Real-time Examples using the Interface Segregation Principle (ISP) in C#.

1. [Library Management System](#)
2. [Restaurant Order Management System](#)
3. [Manage Products in a Store](#)
4. [Animal Tracking System](#)
5. [Document Management System](#)
6. [Entertainment System](#)
7. [Building Management System](#)

How to Use Interface Segregation Principle in C#?

The Interface Segregation Principle (ISP) states that a class should not be forced to implement interfaces it does not use (a class should only implement interfaces it uses). In other words, interfaces should be designed to be specific to the needs of the implementing classes. Following this, the Interface Segregation Principle (ISP) avoids unnecessary coupling between classes and interfaces, resulting in more organized and easier-to-maintain code. This principle prevents unnecessary coupling between classes and interfaces, leading to more maintainable and cohesive code. Here's how to use the Interface Segregation Principle effectively in C#:

- **Identify Client Needs:** Determine the unique behaviors or methods each client class requires for an interface. It's best to avoid creating large, monolithic interfaces that multiple clients will use.
- **Design Fine-Grained Interfaces:** Create smaller, more focused interfaces with methods relevant to each client. This prevents classes from implementing methods they don't use.
- **Avoid Fat Interfaces:** Avoid combining many methods with different purposes when designing interfaces. Instead, break down the interface into smaller interfaces where each serves a specific purpose.
- **Create Specific Interfaces:** Design interfaces for implementing classes' context or domain. This ensures that classes are only required to implement the methods they want.
- **Prefer Composition:** Consider using composition to assemble smaller interfaces and components instead of creating a larger interface covering multiple aspects of a class's behavior.
- **Extract Common Behaviors:** If multiple classes have common methods, extract them into a separate interface to avoid code duplication and promote reuse.
- **Refactor Existing Interfaces:** If any existing interfaces violate the ISP, create smaller, more cohesive interfaces that align with the needs of implementing classes.
- **Use Default Implementations:** In C# 8 and later, default interface implementations can provide basic method implementations. This allows selective method overrides in classes.
- **Apply Dependency Injection:** Implement dependency injection by providing specific interfaces to a class for loose coupling and easy substitution of implementations.

Real-Time Example of Interface Segregation Principle in C#: Library Management System

A library management system can perform multiple tasks, like borrowing books, returning books, and searching the catalog. Users, such as Members, Librarians, and Guests, have different permissions.

- Members can borrow and return books and search the catalog.
- Librarians can manage inventory (add or remove books) and also perform tasks members can.
- Guests can only search the catalog.

Violating ISP:

Let us first see how we can implement the above example without following the Interface Segregation Principle (ISP) in C#. Here's an approach where a single interface tries to encompass all functionalities:

```
using System;
namespace ISPDemo
{
    public class Book
    {
        public string Title { get; set; }
        public string Author { get; set; }
        public string ISBN { get; set; }
    }
    public interface IUser
    {
        void BorrowBook(string bookId);
        void ReturnBook(string bookId);
        void SearchCatalog(string searchTerm);
        void AddBook(Book book);
        void RemoveBook(string bookId);
    }
    public class Guest : IUser
    {
        public void BorrowBook(string bookId)
        {
            throw new NotImplementedException("Guests cannot borrow books.");
        }
        public void ReturnBook(string bookId)
        {
            throw new NotImplementedException("Guests cannot return books.");
        }
        public void SearchCatalog(string searchTerm)
        {
            // Implementation to search books.
        }
        public void AddBook(Book book)
        {
            throw new NotImplementedException("Guests cannot add books.");
        }
        public void RemoveBook(string bookId)
        {
            throw new NotImplementedException("Guests cannot remove books.");
        }
    }
}
```

```

    }
}
}

```

In this approach, the Guest class is forced to implement irrelevant methods.

With to ISP:

Let us see how we can rewrite the above example following the Interface Segregation Principle (ISP) in C#. By segregating the interfaces based on functionality, we can make the system more focused and logical:

```

using System;
namespace ISPDemo
{
    public class Book
    {
        public string BookId { get; set; }
        public string Title { get; set; }
        public string Author { get; set; }
        public string ISBN { get; set; }
        public override string ToString()
        {
            return $"[BookId: {BookId}, Title: {Title}, Author:{Author}, ISBN:
        }
    }
    public interface IBorrowReturn
    {
        void BorrowBook(string bookId);
        void ReturnBook(string bookId);
    }
    public interface ISearchable
    {
        void SearchCatalog(string searchTerm);
    }
    public interface IManageInventory
    {
        void AddBook(Book book);
        void RemoveBook(string bookId);
    }
    public class Member : IBorrowReturn, ISearchable
    {
        public void BorrowBook(string bookId)
        {
            // Implementation to borrow a book.
            Console.WriteLine($"Member Borrow Book, BookId: {bookId}");
        }
        public void ReturnBook(string bookId)
        {
            // Implementation to return a book.
            Console.WriteLine($"Member Return Book, BookId: {bookId}");
        }
    }
}

```

```

    }
    public void SearchCatalog(string searchTerm)
    {
        // Implementation to search books.
        Console.WriteLine($"Member Search Book, Search Catalog: {searchTerm}");
    }
}
public class Librarian : IBorrowReturn, ISearchable, IManageInventory
{
    public void BorrowBook(string bookId)
    {
        // Implementation to borrow a book.
        Console.WriteLine($"Librarian Borrow Book, BookId: {bookId}");
    }
    public void ReturnBook(string bookId)
    {
        // Implementation to return a book.
        Console.WriteLine($"Librarian Return Book, BookId: {bookId}");
    }
    public void SearchCatalog(string searchTerm)
    {
        // Implementation to search books.
        Console.WriteLine($"Librarian Search Book, Search Catalog: {searchTerm}");
    }
    public void AddBook(Book book)
    {
        // Implementation to add a book.
        Console.WriteLine($"Librarian Add Book, {book}");
    }
    public void RemoveBook(string bookId)
    {
        // Implementation to remove a book.
        Console.WriteLine($"Librarian Remove Book, BookId: {bookId}");
    }
}
public class Guest : ISearchable
{
    public void SearchCatalog(string searchTerm)
    {
        // Implementation to search books.
        Console.WriteLine($"Guest Search Book, Search Catalog: {searchTerm}");
    }
}

//Testing the Interface Segregation Principle
public class Program
{
    public static void Main()
    {
        Console.WriteLine("Librarian:");
        Librarian librarian = new Librarian();
    }
}

```

```

Book book = new Book()
{
    BookId = "BK-10001",
    Title = "SOLID Principle using C#",
    Author = "Pranaya Rout",
    ISBN = "ISBN-Demo"
};
librarian.AddBook(book);
librarian.BorrowBook(book.BookId);
librarian.SearchCatalog("SOLID");
librarian.ReturnBook(book.BookId);
librarian.RemoveBook(book.BookId);
Console.WriteLine("\nMember:");
Member member = new Member();
//member.AddBook(book); //Compile Time Error
member.BorrowBook(book.BookId);
member.SearchCatalog("SOLID");
member.ReturnBook(book.BookId);
//member.RemoveBook(book.BookId); //Compile Time Error
Console.WriteLine("\nMember:");
Guest guest = new Guest();
//guest.AddBook(book); //Compile Time Error
//guest.BorrowBook(book.BookId); //Compile Time Error
guest.SearchCatalog("SOLID");
//guest.ReturnBook(book.BookId); //Compile Time Error
//guest.RemoveBook(book.BookId); //Compile Time Error
Console.ReadKey();
    }
}
}

```

By adhering to the Interface Segregation Principle, we now have more fine-grained interfaces that only expose relevant methods to classes. Each user role only needs to implement the tasks that pertain to its responsibilities. When you run the above code, you will get the following output.

```

Librarian:
Librarian Add Book, [BookId: BK-10001, Title: SOLID Principle using C#, Author:Pranaya Rout, ISBN:ISBN-Demo]
Librarian Borrow Book, BookId: BK-10001
Librarian Search Book, Search Catalog: SOLID
Librarian Return Book, BookId: BK-10001
Librarian Remove Book, BookId: BK-10001

Member:
Member Borrow Book, BookId: BK-10001
Member Search Book, Search Catalog: SOLID
Member Return Book, BookId: BK-10001

Member:
Guest Search Book, Search Catalog: SOLID

```

Real-Time Example of Interface Segregation Principle in C#: Restaurant Order Management System

Imagine a restaurant system with different entities like `OnlineOrder`, `InHouseOrder`, and `PhoneOrder`. Each entity has specific actions:

- `OnlineOrder` can process payment online and generate a receipt.
- `InHouseOrder` can print a ticket.
- `PhoneOrder` can confirm the order over the phone.

Violating ISP:

Creating a universal interface for all order types leads to unnecessary implementations. Let us first see how we can implement the above example without following the Interface Segregation Principle (ISP) in C#. Here's an approach where a single interface tries to encompass all functionalities:

```
using System;
namespace ISPDemo
{
    public interface IOrder
    {
        void ProcessOnlinePayment();
        void PrintTicket();
        void ConfirmOverPhone();
    }
    public class OnlineOrder : IOrder
    {
        public void ProcessOnlinePayment()
        {
            // Implementation for processing online payment.
        }
        public void PrintTicket()
        {
            throw new NotImplementedException("Online orders do not print tickets.");
        }
        public void ConfirmOverPhone()
        {
            throw new NotImplementedException("Online orders do not confirm orders over the phone.");
        }
    }
}
```

`OnlineOrder` is forced to implement unnecessary methods in this approach, violating ISP.

With to ISP:

Let us see how we can rewrite the above example following the Interface Segregation Principle (ISP) in C#. By segregating the interfaces, we can make the system more modular and avoid unnecessary implementations:

```
using System;
namespace ISPDemo
```

```

{
    public interface IOnlineOrder
    {
        void ProcessOnlinePayment();
        void GenerateReceipt();
    }
    public interface IInHouseOrder
    {
        void PrintTicket();
    }
    public interface IPhoneOrder
    {
        void ConfirmOverPhone();
    }
    // Implementing segregated interfaces
    public class OnlineOrder : IOnlineOrder
    {
        public void ProcessOnlinePayment()
        {
            // Implementation for processing online payment.
            Console.WriteLine("Processing Online Payment");
        }
        public void GenerateReceipt()
        {
            // Implementation for generating a receipt.
            Console.WriteLine("Generating Receipt");
        }
    }
    public class InHouseOrder : IInHouseOrder
    {
        public void PrintTicket()
        {
            // Implementation for printing a ticket.
            Console.WriteLine("Printing Ticket");
        }
    }
    public class PhoneOrder : IPhoneOrder
    {
        public void ConfirmOverPhone()
        {
            // Implementation to confirm order over the phone.
            // Implementation for printing a ticket.
            Console.WriteLine("Confirming order over the phone");
        }
    }

    //Testing the Interface Segregation Principle
    public class Program
    {
        public static void Main()
        {

```

```

        Console.WriteLine("OnlineOrder:");
        OnlineOrder onlineOrder = new OnlineOrder();
        onlineOrder.ProcessOnlinePayment();
        onlineOrder.GenerateReceipt();
        Console.WriteLine("\nInHouseOrder:");
        InHouseOrder inHouseOrder = new InHouseOrder();
        inHouseOrder.PrintTicket();
        Console.WriteLine("\nPhoneOrder:");
        PhoneOrder phoneOrder = new PhoneOrder();
        phoneOrder.ConfirmOverPhone();
        Console.ReadKey();
    }
}

```

With this design approach, each class only implements the interfaces relevant to its behavior, adhering to the Interface Segregation Principle. This helps in maintaining a cleaner and more intuitive codebase.

Real-Time Example of Interface Segregation Principle in C#: Manage Products in a Store

Imagine you have an online store system where you manage products. Not all products are the same; some are downloadable, like e-books, while others are physical products, like clothes.

Violating ISP

Let us first see how we can implement the above example without following the Interface Segregation Principle (ISP) in C#:

```

using System;
namespace ISPDemo
{
    public interface IProduct
    {
        void SetPrice(decimal price);
        void GetDescription();
        byte[] Download(); // This does not apply to all products!
        void ShipTo(string address); // This does not apply to downloadable pr
    }
    public class Ebook : IProduct
    {
        public void SetPrice(decimal price)
        {
            // Implementation
        }
        public void GetDescription()
        {
            // Implementation
        }
    }
}

```

```

        public byte[] Download()
        {
            byte[] ebook = new byte[1000];
            // Return the e-book file
            return ebook;
        }
        public void ShipTo(string address)
        {
            throw new NotImplementedException("Ebooks cannot be shipped!");
        }
    }
}

```

In the above example, the Ebook class is forced to implement the ShipTo method, which doesn't make sense for e-books.

With to ISP

Let's segregate the interfaces based on the product type. Let us see how we can rewrite the above example following the Interface Segregation Principle (ISP) in C#:

```

using System;
namespace ISPDemo
{
    public interface IProduct
    {
        void SetPrice(decimal price);
        void GetDescription();
    }
    public interface IDownloadable
    {
        byte[] Download();
    }
    public interface IShippable
    {
        void ShipTo(string address);
    }
    public class Ebook : IProduct, IDownloadable
    {
        public void SetPrice(decimal price)
        {
            // Implementation
            Console.WriteLine($"Price Set EBook: {price}");
        }
        public void GetDescription()
        {
            // Implementation
            Console.WriteLine("Get Description : EBook");
        }
        public byte[] Download()

```

```

        {
            Console.WriteLine("Download EBook");
            // Return the e-book file
            return new byte[1000];
        }
    }
}
public class TShirt : IProduct, IShippable
{
    public void SetPrice(decimal price)
    {
        // Implementation
        Console.WriteLine($"Price Set TShirt: {price}");
    }
    public void GetDescription()
    {
        // Implementation
        Console.WriteLine("Get Description : TShirt");
    }
    public void ShipTo(string address)
    {
        // Ship the t-shirt to the given address
        Console.WriteLine($"TShirt ShipTo : {address}");
    }
}
//Testing the Interface Segregation Principle
public class Program
{
    public static void Main()
    {
        Ebook ebook = new Ebook();
        ebook.SetPrice(1000);
        ebook.GetDescription();
        ebook.Download();
        TShirt shirt = new TShirt();
        shirt.SetPrice(2000);
        shirt.GetDescription();
        shirt.ShipTo("Address 1, City, Pindcode");
        Console.ReadKey();
    }
}
}

```

Now, with the segregated interfaces, each product class only implements the methods relevant to its type. The Ebook class no longer has to deal with the irrelevant ShipTo method, and the system becomes clearer and easier to maintain. When you run the above code, you will get the following output.

```
Price Set EBook: 1000
Get Description : EBook
Download EBook
Price Set TShirt: 2000
Get Description : TShirt
TShirt ShipTo : Address 1, City, Pindcode
```

Real-Time Example of Interface Segregation Principle in C#: Animal Tracking System

Let's consider a real-world analogy related to animals. Assume we are developing an animal tracking system where different types of animals have various behaviors. Some animals can fly, swim, or walk.

Violating ISP:

Let us first see how we can implement the above example without following the Interface Segregation Principle (ISP) in C#. If we create a generic IAnimal interface to support all these behaviors, then we would end up having the following design:

```
using System;
namespace ISPDemo
{
    public interface IAnimal
    {
        void Walk();
        void Swim();
        void Fly();
    }
    public class Duck : IAnimal
    {
        public void Walk()
        {
            Console.WriteLine("Duck is walking.");
        }
        public void Swim()
        {
            Console.WriteLine("Duck is swimming.");
        }
        public void Fly()
        {
            Console.WriteLine("Duck is flying.");
        }
    }
    public class Cat : IAnimal
    {
        public void Walk()
        {
            Console.WriteLine("Cat is walking.");
        }
    }
}
```

```

        public void Swim()
        {
            throw new NotImplementedException("Cats usually don't swim.");
        }
        public void Fly()
        {
            throw new NotImplementedException("Cats can't fly.");
        }
    }
}

```

Here, we see a clear violation of ISP. The Cat class has to provide implementations for methods that don't relate to it, like Fly and Swim.

With to ISP:

By segregating the interfaces, we can make the system more modular. Let us see how we can rewrite the above example following the Interface Segregation Principle (ISP) in C#:

```

using System;
namespace ISPDemo
{
    public interface IWalker
    {
        void Walk();
    }
    public interface ISwimmer
    {
        void Swim();
    }
    public interface IFlyer
    {
        void Fly();
    }
    public class Duck : IWalker, ISwimmer, IFlyer
    {
        public void Walk()
        {
            Console.WriteLine("Duck is walking.");
        }
        public void Swim()
        {
            Console.WriteLine("Duck is swimming.");
        }
        public void Fly()
        {
            Console.WriteLine("Duck is flying.");
        }
    }
    public class Cat : IWalker

```

```

{
    public void Walk()
    {
        Console.WriteLine("Cat is walking.");
    }
}

//Testing the Interface Segregation Principle
public class Program
{
    public static void Main()
    {
        Duck duck = new Duck();
        duck.Walk();
        duck.Swim();
        duck.Fly();
        Cat cat = new Cat();
        cat.Walk();
        //cat.Swim(); //Compile Time Error
        //cat.Fly();  //Compile Time Error
        Console.ReadKey();
    }
}

```

Now, each class only needs to implement the interfaces that apply to its behavior. This design adheres to the Interface Segregation Principle, making the system cleaner and more maintainable. When you run the above code, you will get the following output.

```

Duck is walking.
Duck is swimming.
Duck is flying.
Cat is walking.

```

Real-Time Example of Interface Segregation Principle in C#: Document Management System

Let's consider another real-time scenario of a document management system where users can create, read, and update documents.

Violating ISP

Let us first see how we can implement the above example without following the Interface Segregation Principle (ISP) in C#. Suppose we design a single interface for managing documents:

```

using System;
namespace ISPDemo
{

```

```

public interface IDocumentManagement
{
    void CreateDocument(string content);
    string ReadDocument(int id);
    void UpdateDocument(int id, string content);
    void DeleteDocument(int id);
}

public class ReadOnlyUser : IDocumentManagement
{
    public void CreateDocument(string content)
    {
        throw new NotImplementedException("Read-only users cannot create documents.");
    }
    public string ReadDocument(int id)
    {
        // Implementation to read the document.
        return "Sample document content.";
    }
    public void UpdateDocument(int id, string content)
    {
        throw new NotImplementedException("Read-only users cannot update documents.");
    }
    public void DeleteDocument(int id)
    {
        throw new NotImplementedException("Read-only users cannot delete documents.");
    }
}

```

In this approach, a `ReadOnlyUser` has methods it will never use and that doesn't apply to its role, which violates the ISP.

With to ISP

We can break the interface into smaller and more focused interfaces to adhere to the Interface Segregation Principle. Let us see how we can rewrite the above example following the Interface Segregation Principle (ISP) in C#:

```

using System;
namespace ISPDemo
{
    public interface ICreateDocument
    {
        void CreateDocument(string content);
    }
    public interface IReadDocument
    {

```

```

        string ReadDocument(int id);
    }
    public interface IUpdateDocument
    {
        void UpdateDocument(int id, string content);
    }
    public interface IDeleteDocument
    {
        void DeleteDocument(int id);
    }
    // For read-only users:
    public class ReadOnlyUser : IReadDocument
    {
        public string ReadDocument(int id)
        {
            // Implementation to read the document.
            return "Sample Document Content.";
        }
    }
    // For admin users who have all privileges:
    public class AdminUser : ICreateDocument, IReadDocument, IUpdateDocument,
IDeleteDocument
    {
        public void CreateDocument(string content)
        {
            // Implementation to create a document.
            Console.WriteLine($"Create Document: {content}");
        }
        public string ReadDocument(int id)
        {
            // Implementation to read the document.
            Console.WriteLine($"Read Document: Id: {id}");
            return "Sample document content.";
        }
        public void UpdateDocument(int id, string content)
        {
            // Implementation to update the document.
            Console.WriteLine($"Update Document: Id: {id}, content: {content}");
        }
        public void DeleteDocument(int id)
        {
            // Implementation to delete the document.
            Console.WriteLine($"Delete Document: Id: {id}");
        }
    }

    //Testing the Interface Segregation Principle
    public class Program
    {
        public static void Main()
        {

```

```

        AdminUser adminUser = new AdminUser();
        adminUser.CreateDocument("Text Document");
        adminUser.ReadDocument(1);
        adminUser.UpdateDocument(1, "Updating the Content");
        adminUser.DeleteDocument(1);
        ReadOnlyUser readOnlyUser = new ReadOnlyUser();
        readOnlyUser.ReadDocument(1);
        //readOnlyUser.CreateDocument(); //Compile Time Error
        //readOnlyUser.UpdateDocument(); //Compile Time Error
        //readOnlyUser.DeleteDocument(); //Compile Time Error
        Console.ReadKey();
    }
}
}

```

Now, each class only implements the interfaces relevant to its role, ensuring that the system is modular, clear, and avoids unnecessary bloat.

Real-Time Example of Interface Segregation Principle in C#: Entertainment System

Imagine a system where you have different types of devices: Television, Radio, and Smart Speaker. These devices can have different capabilities, such as Play, TurnOff, Tune (specific to Radio), and ConnectToWiFi (specific to Smart Speaker).

Violating ISP:

Let us first see how we can implement the above example without following the Interface Segregation Principle (ISP) in C#. In a simple design, we might have an interface that tries to cater to all these devices:

```

using System;
namespace ISPDemo
{
    public interface IEntertainmentDevice
    {
        void Play();
        void TurnOff();
        void Tune(double frequency);
        void ConnectToWiFi(string network);
    }
    public class Television : IEntertainmentDevice
    {
        public void Play()
        {
            Console.WriteLine("TV Playing...");
        }
        public void TurnOff()
        {
            Console.WriteLine("TV turned off.");
        }
        public void Tune(double frequency)

```

```

        {
            throw new NotImplementedException("TV doesn't support tuning.");
        }
        public void ConnectToWiFi(string network)
        {
            // Connect TV to Wi-Fi network
        }
    }
}

```

This design forces the Television class to implement the Tune method, which is not applicable.

With to ISP:

Let us see how we can rewrite the above example following the Interface Segregation Principle (ISP) in C#. Let's segregate the interfaces according to the functionalities of the devices:

```

using System;
namespace ISPDemo
{
    public interface IPlayable
    {
        void Play();
    }
    public interface ITurnOff
    {
        void TurnOff();
    }
    public interface ITunable
    {
        void Tune(double frequency);
    }
    public interface IWifiConnectable
    {
        void ConnectToWiFi(string network);
    }
    // Television only needs to play, turn off, and connect to WiFi.
    public class Television : IPlayable, ITurnOff, IWifiConnectable
    {
        public void Play()
        {
            Console.WriteLine("TV Playing...");
        }
        public void TurnOff()
        {
            Console.WriteLine("TV turned off.");
        }
        public void ConnectToWiFi(string network)
        {
            Console.WriteLine("Connect TV to Wi-Fi network.");
        }
    }
}

```

```

    }
}
// Radio can play, turn off, and tune.
public class Radio : IPlayable, ITurnOff, ITunable
{
    public void Play()
    {
        Console.WriteLine("Radio Playing...");
    }
    public void TurnOff()
    {
        Console.WriteLine("Radio turned off.");
    }
    public void Tune(double frequency)
    {
        Console.WriteLine($"Radio tuned to {frequency} MHz.");
    }
}
// Smart Speaker can play, turn off, and connect to WiFi.
public class SmartSpeaker : IPlayable, ITurnOff, IWifiConnectable
{
    public void Play()
    {
        Console.WriteLine("Smart Speaker Playing...");
    }
    public void TurnOff()
    {
        Console.WriteLine("Smart Speaker turned off.");
    }
    public void ConnectToWiFi(string network)
    {
        Console.WriteLine("Connect Smart Speaker to Wi-Fi network.");
    }
}

//Testing the Interface Segregation Principle
public class Program
{
    public static void Main()
    {
        Television television = new Television();
        television.Play();
        television.ConnectToWiFi("MyWifi");
        television.TurnOff();
        Radio radio = new Radio();
        radio.Play();
        radio.Tune(91.4);
        radio.TurnOff();
        SmartSpeaker smartSpeaker = new SmartSpeaker();
        smartSpeaker.Play();
        smartSpeaker.ConnectToWiFi("MyWifi");
    }
}

```

```

        smartSpeaker.TurnOff();
        Console.ReadKey();
    }
}
}

```

Now, each device type only needs to implement the interfaces relevant to its functionality, making the design more modular and clearer. This approach adheres to the Interface Segregation Principle. When you run the above code, you will get the following output.

```

TV Playing...
Connect TV to Wi-Fi network.
TV turned off.
Radio Playing...
Radio tuned to 91.4 MHz.
Radio turned off.
Smart Speaker Playing...
Connect Smart Speaker to Wi-Fi network.
Smart Speaker turned off.

```

Real-Time Example of Interface Segregation Principle in C#: Building Management System

Imagine a building management system with various types of rooms: Apartment, Office, and Storage Room. These rooms can have different functionalities:

- Apartments and offices might need cleaning.
- Offices might need a reservation system.
- Storage rooms might need inventory checks.

Violating ISP:

Let us first see how we can implement the above example without following the Interface Segregation Principle (ISP) in C#. Here's an approach where a single interface tries to encompass all functionalities:

```

using System;
namespace ISPDemo
{
    public interface IRoom
    {
        void Clean();
        void Reserve(DateTime from, DateTime to);
        void CheckInventory();
    }
    public class Apartment : IRoom
    {

```

```

        public void Clean()
        {
            Console.WriteLine("Apartment cleaned.");
        }
        public void Reserve(DateTime from, DateTime to)
        {
            throw new NotImplementedException("Apartments can't be reserved.");
        }
        public void CheckInventory()
        {
            throw new NotImplementedException("Apartments don't have inventory");
        }
    }
}

```

This design forces the Apartment class to implement methods that don't pertain to it, which violates ISP.

With to ISP:

Let us see how we can rewrite the above example following the Interface Segregation Principle (ISP) in C#. By segregating the interfaces based on functionality, we can make the system more focused and logical:

```

using System;
namespace ISPDemo
{
    public interface ICleanable
    {
        void Clean();
    }
    public interface IReservable
    {
        void Reserve(DateTime from, DateTime to);
    }
    public interface IInventoryCheck
    {
        void CheckInventory();
    }
    // Apartments can only be cleaned.
    public class Apartment : ICleanable
    {
        public void Clean()
        {
            Console.WriteLine("Apartment cleaned.");
        }
    }
    // Offices can be cleaned and reserved.
    public class Office : ICleanable, IReservable
    {
        public void Clean()
        {

```

```

        Console.WriteLine("Office cleaned.");
    }
    public void Reserve(DateTime from, DateTime to)
    {
        Console.WriteLine($"Office reserved from {from.Date} to {to.Date}.
    }
}
// Storage rooms require inventory checks.
public class StorageRoom : IInventoryCheck
{
    public void CheckInventory()
    {
        Console.WriteLine("Inventory checked in storage room.");
    }
}

//Testing the Interface Segregation Principle
public class Program
{
    public static void Main()
    {
        Apartment apartment = new Apartment();
        apartment.Clean();
        Office office = new Office();
        office.Clean();
        office.Reserve(DateTime.Now, DateTime.Now.AddDays(2));
        StorageRoom storageRoom = new StorageRoom();
        storageRoom.CheckInventory();
        Console.ReadKey();
    }
}
}

```

With the ISP-adherent approach, each room type only implements the interfaces related to its relevant functionalities, making the system more cohesive and understandable.

In the next article, I will discuss the [Dependency Inversion Principle in C#](#) with Examples. Here, in this article, I try to explain **Real-Time Examples of the Interface Segregation Principle (ISP) in C#**. I hope you enjoy this Interface Segregation Principle (ISP) Real-Time Examples using the C# article.